

Chapter Overview

Chapter 14 covers system design principles applied to ShopVerse: scalability patterns, CAP theorem, consistency models, load balancing, rate limiting, circuit breaker, event-driven architecture, CQRS, saga pattern, API gateway design, capacity planning, and disaster recovery strategies.

Sub-Chapter	Topic
14-01	Scalability – vertical vs horizontal, stateless services
14-02	CAP Theorem – consistency, availability, partition tolerance
14-03	Consistency Models – strong, eventual, causal, read-your-writes
14-04	Load Balancing – algorithms, sticky sessions, health checks
14-05	Rate Limiting – token bucket, leaky bucket, sliding window counter
14-06	Circuit Breaker – Resilience4j states, fallback, half-open
14-07	Event-Driven Architecture – choreography vs orchestration
14-08	CQRS – Command Query Responsibility Segregation
14-09	Saga Pattern – distributed transaction management
14-10	API Gateway – routing, auth, rate limit, observability
14-11	Capacity Planning – back-of-envelope calculations
14-12	Disaster Recovery – RPO, RTO, backup strategies

Dimension	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Approach	Bigger machine (more CPU/RAM)	More machines; same-spec nodes
Limit	Single machine ceiling (hardware)	Practically unlimited
Failure mode	Single point of failure	N-1 nodes still serve traffic
Cost	Exponential (premium hardware)	Linear (commodity hardware)
State	Easy – shared memory	Hard – stateless required (or distributed state)
ShopVerse action	DB primary (bigger instance)	API services, Kafka, ES, Redis

Stateless Service Design

ShopVerse Stateless Service Principles

No in-process session state – sessions in Redis

Any instance handles any request – load balancer distributes freely

No in-process cache pinned to instance – cache in Redis

Horizontal scale add/remove instances without data loss

Configuration via environment variables (12-factor app)

Same image, different env = different environment (dev/prod)

Health check endpoint GET /actuator/health returns 200

Load balancer only routes to healthy instances

CAP: a distributed system can guarantee at most 2 of: Consistency (C), Availability (A), Partition Tolerance (P). Since partitions always occur in real networks, the trade-off is CP vs AP.

System	CAP Choice	Behaviour on Partition	ShopVerse Use
PostgreSQL (single primary)	CP	Replica lag; reads may be stale	Order data – must be consistent
Redis Cluster	AP	Split-brain risk; accepts writes	Cart, sessions – availability > consistency
Cassandra	AP	Returns potentially stale data	Product ratings, analytics
MongoDB (replica set w/ majority write)	CP (w/ majority write)	Primary election; brief unavailability	Order events if used
Elasticsearch	AP	Searches may miss recent writes	Product search – eventual is fine

PACELC Extension

Scenario	Trade-off
No partition (normal operation)	PACELC asks: trade Latency vs Consistency
Low latency reads (product page)	Accept eventual consistency: read from ES replica
Order placement (financial)	Require consistency: write to Postgres primary, read-your-writes

Model	Guarantee	Example	ShopVerse Use
Strong consistency	Read sees latest committed write	Single DB node read-after-write	Order status read after payment
Linearisability	Operations appear instantaneous globally	Replica with WAL log commands	Flash sale stock count
Eventual consistency	All replicas converge given no new writes	Cassandra default	Product view counts
Causal consistency	Causally related writes seen in order	MongoDB causal sessions	Comment replies visible after comment
Read-your-writes	You always see your own writes	Session pinning or primary read	Cart updates visible immediately

```
// Read-your-writes in Spring: route writes to primary, reads to replica
// application.yml
spring:
  datasource:
    primary: jdbc:postgresql://primary:5432/shopverse
    replica: jdbc:postgresql://replica:5432/shopverse
    // @Transactional(readOnly=true) routes to replica
    // @Transactional routes to primary
```

Algorithm	How	Best For	ShopVerse
Round Robin	Requests distributed evenly in rotation	Equal capacity instances	Default for product-service pods
Least Connections	Route to instance with fewest open connections	Long running requests	Order-service (long DB transactions)
IP Hash	Hash client IP → always same instance	Session affinity needed	Avoid if using Redis sessions
Weighted Round Robin	Proportion by instance capacity	Mixed capacity instances	Rarely needed in K8s

```
# Nginx upstream load balancing
upstream product_service {
    least_conn;
    server product-svc-1:8080;
    server product-svc-2:8080;
    server product-svc-3:8080;
    keepalive 32;
}
```

Health Check

Check Type	Endpoint	Fail Action
Liveness	GET /actuator/health/liveness	Kubernetes restarts pod
Readiness	GET /actuator/health/readiness	K8s removes from Service endpoints; no traffic
Startup probe	GET /actuator/health	Delays liveness checks; prevents premature kill during startup

Algorithm	Behaviour	Burst Allowed	Spring Use
Token Bucket	Tokens refill at fixed rate; burst up to bucket size	Yes	ProductSearch: 100 req/min; burst 200
Leaky Bucket	Requests drip out at fixed rate; queue overflow drops	No	OrderPlacement: smooth 10 req/s
Sliding Window Counter	Rolling time window; precise count	No	Checkout: 5 attempts per 15 min window
Fixed Window Counter	Count resets at interval boundary	Boundary burst	Login: 10 attempts/hour

```
// Resilience4j rate limiter - 100 calls per 60 seconds
RateLimiterConfig config = RateLimiterConfig.custom()
    .limitForPeriod(100)
    .limitRefreshPeriod(Duration.ofMinutes(1))
    .timeoutDuration(Duration.ofMillis(100))
    .build();

// Spring Boot via @RateLimiter annotation
@RateLimiter(name = "productSearch", fallbackMethod = "searchFallback")
@GetMapping("/search")
public List<ProductDto> search(@RequestParam String q) { ... }
public List<ProductDto> searchFallback(String q, RequestNotPermitted e) {
    return cachedResults.getOrDefault(q, Collections.emptyList());
}
```

Circuit Breaker State Machine

Circuit Breaker States

CLOSED – normal operation; requests flow through

Failure rate threshold: if >50% failures in last 10 calls → open

OPEN – all calls rejected immediately (fail fast)

Wait duration: 10s → then transition to HALF_OPEN

HALF_OPEN – allow N test calls through

If test calls succeed: transition to CLOSED

If test calls fail: transition back to OPEN

```
@CircuitBreaker(name="inventoryService", fallbackMethod="fallback")
@Retry(name="inventoryService")
@TimeLimiter(name="inventoryService")
public CompletableFuture<Integer> getStock(Long productId) {
    return CompletableFuture.supplyAsync(
        () -> inventoryClient.getStock(productId));
}

public CompletableFuture<Integer> fallback(Long productId, Exception e) {
    return CompletableFuture.completedFuture(
        stockCache.getOrDefault(productId, -1)); // -1 = unknown
}
```

Config	Value	Meaning
slidingWindowSize	10	Count failures across last 10 calls
failureRateThreshold	50	Open if 50%+ fail
waitDurationInOpenState	10s	Stay OPEN for 10s before HALF_OPEN
permittedNumberOfCallsInHalfOpenState	5	5 probe calls in HALF_OPEN state

Pattern	Coordination	Pros	Cons
Choreography	Each service listens to events, once triggered, reacts to add functionality	Low coupling, easy to add functionality	Hard to trace full flow; coupling via events
Orchestration	Central saga coordinator (Orchestrator) sends business logic to participants	Single point of failure	Orchestrator = single point of failure

ShopVerse Order Placement – Choreography

OrderService publishes OrderPlacedEvent

InventoryService listens → reserves stock → publishes StockReservedEvent

Success: OrderService listens → sets status=CONFIRMED

Failure: publishes StockFailedEvent → OrderService cancels order

PaymentService listens to StockReservedEvent → charges card

Success: publishes PaymentSucceededEvent

Failure: publishes PaymentFailedEvent → InventoryService releases stock

NotificationService listens to PaymentSucceededEvent → sends email

LoyaltyService listens to PaymentSucceededEvent → adds points

CQRS: separate the write model (Commands that modify state) from the read model (Queries that return data). Allows independent scaling, optimised read schemas, and different storage for reads vs writes.

ShopVerse CQRS – Product Service

Write Side (Command)

CreateProductCommand, UpdatePriceCommand, DeactivateProductCommand

Writes to: PostgreSQL (source of truth)

Publishes ProductChangedEvent to Kafka

Read Side (Query)

ProductSearchQuery, ProductDetailQuery, CategoryListingQuery

Reads from: Elasticsearch (product search), Redis (product detail cache)

Kafka consumer keeps read models in sync with write model

	Write Side	Read Side
Storage	PostgreSQL (normalised)	Elasticsearch + Redis (denormalised)
Consistency	Strong (ACID)	Eventual (CDC lag 50-200ms)
Optimisation	Write throughput; constraint checking	Read throughput; search; aggregation
Scaling	Scale with writes; smaller	Scale independently; larger typically

Sagas replace 2-phase commit in microservices. Each saga step has a compensating transaction that undoes the step if a later step fails.

Saga Step	Action	Compensating Transaction
1. Reserve stock	inventory.reserve(productId, qty)	inventory.release(productId, qty)
2. Charge payment	payment.charge(customerId, amount)	payment.refund(customerId, amount)
3. Create order	order.create(orderDetails)	order.cancel(orderId)
4. Send confirmation	notification.sendEmail(orderId)	notification.sendCancellation(orderId)

```
// Spring State Machine – OrderSaga orchestration
@Component public class OrderSaga {
    @SagaEventHandler(associationProperty = "orderId")
    public void on(OrderPlacedEvent e, @Autowired InventoryCommandGateway inv) {
        inv.send(new ReserveStockCommand(e.getOrderId(), e.getProductId(), e.getQty()));
    }
    @SagaEventHandler(associationProperty = "orderId")
    @EndSaga
    public void on(OrderCompletedEvent e) { /* saga ends */ }
    @SagaEventHandler(associationProperty = "orderId")
    @EndSaga
    public void on(StockReservationFailedEvent e, @Autowired OrderCommandGateway orders) {
        orders.send(new CancelOrderCommand(e.getOrderId())); // compensate
    }
}
```

ShopVerse API Gateway (Spring Cloud Gateway / Nginx)

Routing – path-based routing to microservices

/api/v1/products/** → product-service:8080

/api/v1/orders/** → order-service:8081

/api/v1/users/** → user-service:8082

Authentication & Authorisation – validate JWT at gateway

Reject unauthenticated requests before reaching services

Rate Limiting – per API key / IP / user

Token bucket: 100 req/min anonymous; 1000 req/min authenticated

Observability – distributed tracing (Zipkin), access logs, metrics

TLS Termination – HTTPS at gateway; HTTP internally

Gateway Concern	Spring Cloud Gateway	Nginx
Routing	RouteLocatorBuilder DSL	location + proxy_pass
Rate limiting	RedisRateLimiter (token bucket)	ngx_http_limit_req_module
Auth (JWT)	Spring Security filter	auth_request to auth-service
Circuit breaker	Resilience4j filter	None native; use Lua
Dynamic config	Spring Cloud Config	Reload nginx.conf

ShopVerse Back-of-Envelope

Metric	Estimate	Calculation
Daily active users	500 000	Given; ShopVerse mid-scale
Peak RPS (product search)	~600	$500k \text{ users} \times 10 \text{ searches/day} \div 86400s \times 10\times \text{peak}$
Product record size	~2 KB	Name+desc+price+meta; 10 KB with translations
Total products	5 million	Large catalog
Products storage (Postgres)	10 GB	$5M \times 2 \text{ KB}$
Elasticsearch index size	~25 GB	$5M \times 5 \text{ KB per document (text + metadata)}$
Redis cache (top 10k products)	10 MB	$10k \times 10 \text{ KB per cached object}$
Kafka throughput (orders)	~1 000 events/sec peak	$500k \text{ users} \times 2 \text{ orders/day peak hour}$
Component	Sizing Recommendation	
Product service pods	3 pods $\times$ 2 vCPU $\times$ 4 GB; HPA max 10	
PostgreSQL primary	8 vCPU $\times$ 32 GB; 500 GB SSD; read replica for reporting	
Redis cluster	3 masters $\times$ 3 replicas; 8 GB per node	
Elasticsearch	3 data nodes $\times$ 16 GB; 2 primary shards per index	
Kafka	3 brokers; replication factor 3; 7-day retention	

Term	Definition	ShopVerse Target
RPO (Recovery Point Objective)	Max acceptable data loss duration	< 1 minute (Postgres WAL streaming)
RTO (Recovery Time Objective)	Max acceptable downtime duration	< 5 minutes (automated failover)
MTTR	Mean time to recover	Target < 15 minutes for full service
Layer	DR Strategy	Recovery Time
Postgres	Streaming replication + WAL archive to S3 (promote replica on primary fail)	< 1m (S3 replication lag)
Redis	Redis Sentinel; promote replica; AOF persistence to recover to latest	< 30s
Kafka	Replication factor 3; min.insync.replicas=2; Kafka MirrorMaker to secondary DC	< 2m
Elasticsearch	1 replica per shard; snapshot to S3; ES-ctl	< 30m (restore from snapshot)
Application	Kubernetes deployment; pods auto-rescale; HPA scales on CPU/memory	< 2min
Full region failure	Active-passive DR in secondary region; Route53 DNS failover	< 15min

■ Do

- ✓ Design services stateless – store sessions and cache in Redis so any instance can serve any request.
- ✓ Use circuit breaker (Resilience4j) for all external service calls – fail fast with fallback.
- ✓ Apply rate limiting at the API gateway – protect services from abuse and traffic spikes.
- ✓ Use CQRS for product catalog: writes to Postgres; searches from Elasticsearch via CDC.
- ✓ Implement saga pattern with compensating transactions for multi-service order placement.
- ✓ Define RPO < 1 min and RTO < 5 min; use Postgres streaming replication + Patroni failover.
- ✓ Add readiness and liveness probes to all services – K8s only routes traffic to ready pods.
- ✓ Do back-of-envelope calculations before designing – size storage, throughput, cache needs first.
- ✓ Use eventual consistency for reads (product search, ratings) and strong for writes (orders, stock).
- ✓ Monitor circuit breaker state (open/closed) in dashboards – open state indicates downstream issues.

■ Anti-Patterns

- In-process session state in Spring bean – breaks horizontal scaling; any pod restart loses sessions.
- 2-phase commit across microservices – distributed locking; complex; use saga instead.
- No circuit breaker on external calls – one slow service cascades to full system failure.
- Reading from Postgres replica for order status after placing order – stale read; use primary.
- Large monolithic transaction spanning multiple services – impossible; each service owns its DB.
- No DR plan – single region, no replication, no backups; any AZ failure = data loss.
- Over-engineering for day-1 scale – add microservices/CQRS/saga when needed; start modular monolith.

Sub-Chapter	Key Point	One-Liner
14-01 Scalability	Stateless services	Sessions in Redis; cache in Redis; any pod = any request
14-02 CAP	CP vs AP trade-off	Postgres=CP; Redis=AP; ES=AP; choose per data type
14-03 Consistency	Strong for orders	Eventual for search; strong for financial data
14-04 Load Balancing	Least connections	Nginx upstream least_conn; health check readiness
14-05 Rate Limiting	Token bucket	100 req/min; burst to 200; 429 Too Many Requests
14-06 Circuit Breaker	CLOSED/OPEN/HALF_OPEN	50% fail rate → OPEN; 10s wait → HALF_OPEN probe
14-07 EDA	Choreography for orders	Services react to Kafka events; no central coordinator
14-08 CQRS	Write=Postgres; Read=ES	ES keeps ES in sync; eventual consistency ~200ms
14-09 Saga	Compensating transactions	Each step reversible; no distributed 2PC needed
14-10 API Gateway	Route + auth + rate limit	TLS at gateway; JWT validation; one entry point
14-11 Capacity	1000 peak RPS for search	3 pods × 2 vCPU + HPA; Postgres 32 GB RAM
14-12 DR	RPO < 1 min; RTO < 5 min	Patroni auto-failover; snapshot + WAL to S3

Interview Quick-Check

Question	Concise Answer
CAP theorem trade-offs in ShopVerse	Orders: CP (Postgres) – consistency over availability. Cart: AP (Redis) – availability over consistency.
CQRS benefit?	Separate write model (Postgres ACID) from read model (ES search-optimised). Each scales independently.
Saga vs 2PC?	2PC: distributed lock across services; complex, slow, not microservice-friendly. Saga: local transactions.
Circuit breaker states?	CLOSED (normal). OPEN (fail fast after threshold). HALF_OPEN (probe with N calls). Prevents cascading failures.
Rate limiting algorithm?	Token bucket: allows burst up to bucket size. Leaky bucket: smooth output rate. Sliding window: prevents burst.